

Reveree: Diagnosing LLM Reverse-Engineering Agents

Hadjer Benkraouda
UIUC

Hongyu Cai
Purdue University

Berkay Celik
Purdue University

Gang Wang
UIUC

Abstract

Reverse engineering (RE) is critical to security tasks such as malware analysis and vulnerability discovery, and large language model (LLM) agents are increasingly able to perform it autonomously. Capture-the-flag (CTF) RE challenges have become the standard proxy for measuring this capability, but evaluation rests on a single criterion: whether the agent captures the flag. This solve rate reveals neither *where* in the RE process an agent fails nor *whether* a success reflects analysis of the binary or recall of a public solution. In this paper, we propose REVEREE, a diagnostic framework that scores an LLM RE agent’s trajectory at three tiers: solve rate, milestone progress through an eight-stage RE schema, and a behavioral profile of its actions. Comprehension stages are scored by an outcome-blinded LLM judge validated against a human expert; all other stages are verified deterministically. Using REVEREE, we evaluate nine frontier models and four prompting strategies on 88 picoCTF and NYU-CTF challenges. We find that the base model dominates performance, whereas prompting strategy is a secondary, model-dependent effect. Surprisingly, larger, newer, or costlier models are not reliably stronger. We also find that failures concentrate at the comprehension stages of the RE process, and that extra budget, persistence, or reasoning effort rescues few of them, pointing to a competence limit rather than a resource limit. Regarding memorization, while models reproduce picoCTF flags from challenge descriptions alone, NYU-CTF shows minimal measurable recall, and most solves survive surface perturbation, indicating that genuine analysis coexists with memorization. We release REVEREE to the community.

1 Introduction

Reverse engineering (RE), the recovery of a program’s structure and behavior from its compiled form, is critical to security tasks such as malware analysis, vulnerability discovery, and firmware analysis [7, 18, 21, 33, 45, 56, 67]. The work is expensive and iterative: an analyst alternates between disassemblers, decompilers, debuggers, and scripts to recover

what the code does and how to drive it [9, 10, 53]. Large language models (LLMs) show promise on individual RE sub-tasks [8, 24, 26, 50, 60, 62], and LLM agents, which interleave reasoning with tool use, are increasingly able to carry out the process end to end [1, 19, 44, 52, 72]. How well they do so is therefore a pressing question. The common way to answer it is to use capture-the-flag (CTF) RE challenges as a proxy [1, 42, 43, 51, 52, 55, 64, 68]: the agent is given a binary and must recover a hidden flag, a task that exercises real RE competencies, is machine-checkable, and can be perturbed without changing its difficulty.

Existing evaluations rest almost entirely on a single criterion, the *solve rate*: the fraction of challenges whose flag is captured [1, 42, 51, 52, 64, 68]. For reverse engineering, this criterion is necessary but insufficient. It floors on hard challenges, where every model fails and diagnosis matters most. It cannot localize failure: a run that recovered the binary’s core algorithm and stalled one step short of a solver scores the same as one that never parsed the file, although the two call for different fixes. And it cannot separate *reasoning from recall*: public challenges and their write-ups leak into pre-training, so a captured flag may be reproduced from memory rather than derived from the binary, a confound that grows as models train on more of the web content.

Goal and Method. In this paper, we develop REVEREE, a diagnostic framework for evaluating LLM RE agents. Its key idea is to score the agent’s trajectory rather than its outcome alone, at three tiers: 1) the solve rate, 2) milestone progress through an eight-stage schema of the RE process. Stages that leave a mechanical trace are verified deterministically from tool calls; only comprehension stages are referred to an outcome-blinded LLM judge validated against a human expert, and 3) a behavioral profile that models the run as a Markov chain [34] over behavioral states. A memorization probe queries the bare model for a flag from the challenge description alone and re-runs the agent on renamed or paraphrased challenges, holding difficulty fixed. The harness is not our contribution: we build on D-CIPHER [52] and hold it fixed, so that differences in outcome are attributable to the

model or the prompting strategy rather than to the scaffold.

Evaluation. Using REVEREE, we evaluate nine frontier models from the Claude and GPT-5 families, spanning size tiers and versions, under four prompting strategies (vanilla, ReAct [65], Reflexion [46], and a hypothesis ledger) on 88 RE challenges from picoCTF [13] and NYU-CTF [42]. REVEREE exposes five patterns that solve rate hides. First, the base model dominates performance, whereas prompting strategy is model-dependent. Second, larger, newer, or costlier models are not reliably stronger. Third, failures are not random: on hard challenges, agents recover the structure of the binary and stall at the comprehension stages, and extra budget, persistence, or reasoning effort rescues few of them, pointing to a competence limit rather than a resource limit. Fourth, successful runs share a behavioral signature that failed runs lack, pivoting from analysis into building a solver rather than re-analyzing the binary, and it holds across every model and strategy. Finally, memorization is real but benchmark-dependent: models reproduce picoCTF flags from challenge descriptions alone, whereas NYU-CTF shows minimal measurable recall, and most solves survive surface perturbation, so genuine analysis coexists with memorization. These findings motivate RE-specific reasoning strategies aimed at the comprehension bottleneck (Section 7.2).

Contributions. We make three contributions.

- We propose REVEREE, an evaluation method beyond solve rate, that scores an agent’s trajectory at three tiers and pairs them with a memorization probe. We open-source our tool to the community.
- We perform a controlled evaluation of nine frontier models and four prompting strategies on 88 RE challenges and identify a success-versus-failure signature that replicates across all nine models.
- We carryout a memorization evaluation to separate reasoning from recall with difficulty-preserving perturbations, and show that memorization is benchmark-dependent.

2 Background and Motivation

Reverse engineering (RE). Reverse engineering is the recovery of a program’s structure and behavior from its compiled form, without access to source code. It underpins a range of security tasks, including vulnerability discovery [33, 45], malware analysis [56, 67], and firmware analysis [18]; in many cases the only available artifact to assess is a binary [16, 21]. The work is inherently multi-step and iterative. Empirical studies of professional reverse engineers describe this trajectory: An analyst first *triages* the binary to establish its format, architecture, and rough structure; *maps its surface* through strings, imports, etc.; *recovers control flow* to be able to locate the functions that matter, i.e., *recognizes the core algorithm*

a target software implements. Next, the analyst *extracts the constraints*. These are the conditions that a correct input must satisfy, e.g., a required byte at each position or the input/flag length that produce the desired behavior. Finally, they *construct and run a solver* that produces the answer [9, 10, 54].

Two features of this process make it a stringent test of an autonomous agent. First, it relies on *tool-use*: progress depends on calling disassemblers, decompilers, and debuggers and interpreting their output. Second, it is *hypothesis-driven*, requiring the analyst to form, test, and revise conjectures about what the code does rather than to read an answer off the page. These sub-tasks are increasingly automated: classical binary-analysis frameworks tackle them with symbolic and dynamic techniques [12, 47], and a growing line of work applies LLMs to decompilation, decompiler refinement, and variable- and function-symbol recovery [2, 8, 24, 26, 27, 50, 60, 62], which sharpens the question this paper asks: whether an agent genuinely comprehends a binary or merely pattern-matches its surface.

CTFs as a proxy. Capture-the-flag (CTF) reverse-engineering challenges distill this process into a controlled, verifiable task, and have become a standard proxy for evaluating LLM agents on offensive-security and reverse-engineering tasks [1, 5, 19, 22, 25, 30, 42–44, 51, 52, 61, 64, 68, 71]. The agent is given a binary and a short description and must recover a hidden flag, a secret string typically requiring understanding the binary’s logic, such as a key-validation routine, a custom encoding, or a layer of obfuscation. Three properties make CTF reverse engineering a useful proxy for understanding real-world reverse engineering activities, and account for its wide adoption as an evaluation setting [55]. First, they exercise the same core competencies as real RE: static and dynamic analysis, algorithm recognition, and constraint solving, across a wide range of difficulty. Second, its ground truth is unambiguous and machine-checkable: a submitted flag either validates against the challenge oracle or does not, which makes automated, reproducible evaluation possible without human grading. Third, challenges are *perturbable*: the binary can be modified while its analytic difficulty is held fixed, which is what makes a contamination-controlled experiment feasible. CTF challenges have some limitations; they omit the scale of real firmware and the open-ended goals of a genuine investigation. One property is *double-edged and is central* to our study: many challenges, and detailed write-ups of their solutions, are published online, so strong performance on a documented challenge may reflect recall of its solution rather than analysis of its binary.

LLM Agents. An LLM agent attempts an RE challenge by interleaving reasoning with tool use inside a control loop. Given the binary, it may issue a tool call to, for example, disassemble a function, decompile it, run it under a debugger, or execute a script. Next, it observes the result, updates its understanding, and repeats until it submits a flag or exhausts a budget of turns or tokens. A harness/framework supplies

the scaffold around this loop. Single-agent harnesses give one model a curated tool interface: EnIGMA [1], built on SWE-agent [63], contributes interactive tools for debugging and remote interaction. Multi-agent harnesses decompose the task across roles: D-CIPHER [52], which we adopt, pairs a planner that sets strategy with executors that issue the concrete tool calls. Beyond CTF challenges, agents are also used to automate penetration testing and broader security tasks [19, 61, 72]. Orthogonal to the harness, the agent’s reasoning loop is shaped by general-purpose prompting strategies such as ReAct [65], which interleaves reasoning traces with actions, and Reflexion [46], which adds self-reflection across attempts, among others. Prompting strategies are typically presented as model- and domain-agnostic. Across this landscape, agents are evaluated on CTF benchmarks such as NYU-CTF [42] and almost always by a single number: the solve rate [51].

Our Motivation. The bulk of the work on language-model agents for capture-the-flag and reverse engineering is evaluated almost entirely by solve rate: did the agent submit the correct flag? For reverse engineering this single number is lossy and insufficient, for three reasons that compound. First, *it floors on hard challenges*. A corpus of genuinely difficult binaries drives solve rate toward zero for every model, collapsing the distinctions between models, between strategies, between near-misses and non-starters that an evaluation exists to draw. The harder and more interesting the challenge, the less the solve rate metric reveals. Second, *it cannot locate the cause of failures*. Reverse engineering is a sequence of distinct sub-abilities, and an agent can fail at any of them; yet a run that recovered a function’s algorithm and stalled at constraint extraction earns the same “zero” as a run that never parsed the binary, even though the two runs call for different fixes. Third, *it cannot separate reasoning from recall*. Because challenges and their write-ups leak into pre-training, a captured flag may be reproduced from memory rather than derived from the binary. This confound strengthens as models train on ever more of the public web, so an evaluation that ignores it becomes less trustworthy over time.

These gaps motivate a different kind of evaluation: an instrument that (i) measures partial progress through the RE process, so that hard challenges still discriminate; (ii) localizes where and how an agent fails, so that failures are actionable rather than merely counted; and (iii) separates genuine analysis from solution recall/memorization, so that a reported success can be trusted. The remainder of this paper develops such an instrument, REVEREE, and applies it.

3 Evaluation Pipeline Overview

Our study measures *how* a language-model agent attempts a reverse-engineering challenge by varying a set of factors. Figure 1 shows the pipeline that turns one factor setting into a scored run. A challenge ①, comprising its name, description, and binary files, is paired with a configuration ② that fixes

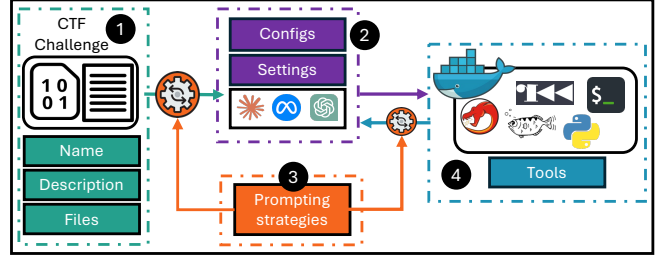


Figure 1: The evaluation pipeline. A reverse-engineering challenge ① (name, description, files) is paired with a model configuration and settings ② and a prompting strategy ③; the configured agent runs in a sandboxed environment with a fixed reverse-engineering toolset ④, producing a trajectory that REVEREE scores. The challenge features, the model and its configurations and settings, and prompting strategies (①, ②, ③) are the varied factors; the execution environment and toolset (④) are held fixed across all runs.

the model and its hyperparameters and with a prompting strategy ③ that determines the agent’s reasoning loop. The configured agent runs inside a sandboxed environment ④ equipped with a standard reverse-engineering toolchain [3, 23, 31], issuing tool calls and observing their output until it submits a flag or exhausts its budget. The resulting trajectory is scored by REVEREE (Section 4); that scored trajectory is the pipeline’s only output and the subject of measurement in this paper.

3.1 Design Goals

We design the pipeline around three goals.

1. **Controlled variation.** To attribute a difference in behavior to a single factor (a model, a prompting strategy, a hyperparameter), every other part of the pipeline must be held fixed. The execution environment and the toolset are therefore identical across runs, so that a difference in outcome reflects the factor under study rather than a change in apparatus.
2. **Framework-agnostic.** The contribution is a methodology, not a harness/framework. We instantiate the pipeline on one agentic harness (D-CIPHER), but the parts that constitute the method do not depend on that harness. Only a simple adapter is needed to convert framework outputs into REVEREE suitable inputs (Section 4.2). Any harness that records its tool calls and its outcome can be substituted without changing what is measured.
3. **A realistic agentic setting.** Agents act in a live environment with the disassemblers, decompilers, debuggers, and scripting tools a human reverse engineer would use, rather than answering questions about a static binary dump. What we measure is the agent’s actual tool use, not a proxy for it.

3.2 The Agentic Framework

The agent is driven by D-CIPHER [52], a multi-agent CTF harness/framework in which a planner directs one or more executors that issue the concrete tool calls. We adopt it as prior-art infrastructure rather than as a contribution: it supplies the agent scaffold, the closed toolset, and the sandboxed environment, and it validates a submitted flag against the challenge oracle. Holding this apparatus fixed is what allows the model and the prompting strategy to be isolated as the variables of interest.

The harness/framework is an *instantiation*, not a *dependency*, of the method. REVEREE consumes a normalized trajectory which is a sequence of (thought, action, observation) steps together with the run’s validated outcome. REVEREE’s schema, two-channel detection, progress metrics, and behavioral profiling operate entirely on that representation. The only harness-specific component is the adapter that produces it (Section 4.2); a different harness, a different toolset, or a single-agent rather than a multi-agent scaffold changes the adapter, but nothing downstream. The diagnostic methodology therefore transfers unchanged, a property that matters precisely because harnesses are evolving quickly.

3.3 Factors of Variation

Within this fixed apparatus, we vary four factors (independently) and hold the rest constant; we state only the high-level design here and defer the concrete settings to the experimental setup (Section 5.1).

Varied. The *model* (②) is the primary axis: we sweep a set of frontier models spanning the leading families and a capability gradient within each. The *prompting strategy* (⑥) varies the agent’s reasoning loop while leaving the harness and tools untouched, isolating the effect of reasoning structure. Selected *challenge features* (e.g., name and description) (①) and *configurations and settings* (e.g., reasoning effort and the budget caps) (②) are varied in targeted ablations. The specific models, strategies, hyperparameter settings, and the composition of the challenge corpus are given in Section 5.1.

Held fixed. The execution environment and its reverse-engineering toolset (④) are identical across runs.

4 REVEREE: A Diagnostic Instrument for Reverse-Engineering Agents

In this section, we introduce REVEREE,¹ a process-level instrument that scores how far an agent advanced through the reverse-engineering process rather than only whether it captured the flag. REVEREE examines each run at three tiers, which answer successively deeper questions. *Tier 1* is the solve rate which measures whether the agent retrieved the

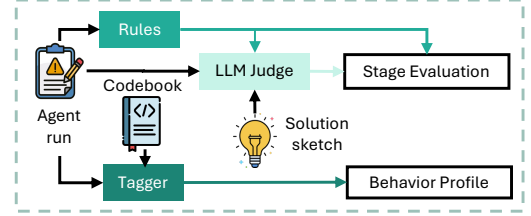


Figure 2: The REVEREE pipeline.

flag. This is the standard outcome metric, which REVEREE retains as the baseline that the remaining tiers explain. *Tier 2* measures progress. Specifically, it assesses how far a run advanced through the reverse engineering stages. *Tier 3* measures behavior: it views a run as a state machine over the agent’s actions and extracts patterns from it.

4.1 Design Principles

Three principles separate REVEREE from prompting an LLM to grade a trajectory: 1) **An RE-specific fixed schema.** Credit is given based on eight canonical reverse-engineering stages, scored uniformly across challenges, rather than a generic competency rubric or hand-authored per-challenge milestones (Section 4.3). 2) **The right tool for each stage.** Deterministic verification is used for mechanical stages such as triage, detecting solver artifacts, and flag validation. Only comprehension stages are referred to an LLM judge (Section 4.4). 3) **Process-centered.** Beyond a partial-credit score, REVEREE reports a stall funnel that localizes where runs fail and analyzes behavioral patterns of agents (Sections 4.5, 4.6).

4.2 Inputs

REVEREE scores runs produced by any CTF agent and consumes each run from three files: First, the *agent run* which records agent output and interactions with tools. For D-CIPHER [52], a multi-agent CTF framework, this includes a *planner message list* and one or more *executor sessions*, *submitted flag* and *success* fields. Second, a *solution sketch* which is a short ground-truth description of how the challenge is solved. This is used to give the judge challenge-specific meanings for each stage but no flag. Third, *run meta-data* which carries identity and configuration information.

4.3 The Eight-Stage RE Schema

The unit of credit is the *stage*, not a hand-authored per-challenge milestone, so no manual milestone labeling is needed; the solution sketch gives challenge-specific stage context. REVEREE fixes the RE stages based on common steps that RE analysts perform as discussed in Section 2. The eight stages are derived from progression documented in observational studies of human RE analysts [9, 10, 54]. The

¹ REVEREE—a referee for reverse-engineering challenges.

Stage	Type	What it denotes
S1 Triage	ACTION	File/format, architecture, packing and protections.
S2 Surface mapping	ACTION	Disassembly/decompilation exposing program structure.
S3 Control flow	INSIGHT	The relevant branching/gating logic is understood.
S4a Algorithm recognize	INSIGHT	Core transformation spotted via a beacon (loop, constant, XOR).
S4b Algorithm name	INSIGHT	Transformation explicitly named (“this is RC4/base64”).
S5 Constraint extract	INSIGHT	The concrete conditions a valid input must satisfy are stated.
S6 Solver	ACTION	A solver/inversion artifact is built (keygen, patch, SMT, angr).
S7 Flag	ACTION	The correct flag is produced and validated.

Table 1: The eight-stage RE schema. ACTION stages leave a mechanical trace and are verified deterministically; INSIGHT stages require demonstrated comprehension and are the only ones the LLM judge rules on.

stages are shown in Table 1 and span the RE milestones from triage to flag validation. Each stage is tagged ACTION (an externally observable operation that leaves a mechanical trace) or INSIGHT (an internal comprehension step), and this tag governs how the stage is detected. For practicality, we split the algorithm stage into *recognize* (S4a) and *name* (S4b), so that spotting a transformation (e.g., a loop XORing against a constant) results in comprehension credit even when the agent never attaches the textbook name. This is a more granular measurement than a single “identified the algorithm” bit.

4.4 Two-Channel Stage Detection

Each stage’s verdict comes from one of two channels. The principle is to use deterministic evidence wherever the evidence is the accomplishment, and to invoke the judge only where the accomplishment is a matter of comprehension.

Channel A. This channel is scored using manually written deterministic rules/scripts that are applied to the executed tool calls and their outputs. For each stage, the tool emits one of confirm, deny, or abstain outcomes. It confirms where the artifact is the accomplishment, such as S7 being confirmed by extracting the flag, S6 by a solver artifact, and S1 by a triage tool having run. Elsewhere when, for example, S2 cannot have occurred if no disassembler or decompiler was ever called, so their absence yields a deterministic negative and Channel A outputs a deny. In cases where there is an uninformative invocation of a tool it (abstains) and defers to Channel B. Channel A also abstains on the pure-comprehension stages S3, S4a, and S4b, S5 which leave no obvious mechanical

trace. The matching invoked by Channel A uses keyword buckets for triage, disassembly/debugging, and solver tools where each is matched against the actual shell commands on whole-word boundaries, with underscores and dots treated as separators. This ensures a tool name embedded in a filename never false-fire, and merely naming a tool or technique in plan prose (“solve,” “invert,” “patch”) cannot confirm a stage. **Channel B.** Channel B is the LLM judge, consulted only for the comprehension stages and only where Channel A abstains, so a run that fails early or solves mechanically may incur zero judge calls. To avoid the pitfalls of using LLM-as-a-judge [58, 69], four constraints separate it from a free-form “grade this transcript” prompt. It is *stage-scoped*: the judge is asked only about the specific stages in question, each defined by the solution sketch, never about generic competencies. It is *outcome-blinded*: the submitted flag, the true flag, and any correct/incorrect marker are redacted before the judge sees the trajectory, so it cannot anchor on “it solved, therefore it understood.” It is *evidence-bound*: every positive finding must cite a concrete trajectory step and a short verbatim quote; if a citation is absent, the stage is not credited. And it is *execution-grounded*: alongside the planner’s reasoning, the judge receives the executor’s real tool calls and outputs. This grounding lets the citation requirement be met against what the agent actually observed.

4.5 Tier-2: Progress Metrics

A worked example. Consider a challenge whose applicable stages are S1–S7, scored for three runs:

Run	Stages reached	Stage	Depth	Outcome
A	S1, S2, S3, S4a	4/8	4/8	unsolved
B	S1, S3, S4a	3/8	1/8	unsolved
C	S1, S2, S6, S7	4/8	2/8	solved

The *stage coverage* counts every stage reached, crediting progress regardless of gaps: Run A reaches four of eight stages (4/8), Run B three (3/8). *Progress depth* counts only the longest unbroken prefix $S1 \rightarrow S2 \rightarrow \dots$ and stops at the first miss: Run A is intact through S4a (4/8), but Run B skips S2, so its depth collapses to S1 alone (1/8) despite reaching S4a. When coverage exceeds depth, e.g., Run B (3/8 vs. 1/8), the run reached later stages without the intermediate ones.

The stall funnel. The *marginal reach* of a stage is the fraction of runs that reached that stage not the fraction that reached at least that far. Each cell is that fraction within a population: for stage S and population P, it is the number of runs in P that reached S divided by $|P|$. Taking the S2 column as a worked cell: two of the three runs (A and C) reached S2, so the “All” entry is 2/3; of the runs, only C is solved and it reached S2, giving 1/1 for “S”; among the two unsolved runs only A reached S2, giving 1/2 for “U”. Applying this to every stage yields:

	S1	S2	S3	S4a	S4b	S5	S6	S7
All	3/3	2/3	2/3	2/3	0/3	0/3	1/3	1/3
S	1/1	1/1	0/1	0/1	0/1	0/1	1/1	1/1
U	2/2	1/2	2/2	2/2	0/2	0/2	0/2	0/2

A cumulative curve would read 1 at every stage for the solved run C (because it reached S7), hiding that C skipped S3–S5. The marginal view exposes this. The “solved-without-understanding” or “shortcut solve” signature of a flag produced without the underlying analysis, while the unsolved row shows the opposite (comprehension reached, no solver or flag). Table 8 shows this contrast at corpus scale on picoCTF.

4.6 Tier-3: Behavioral Profiling

The milestone tiers characterize *what* an agent accomplished; they do not clarify *how* it spent its effort. This procedural dimension is central to reverse engineering, where progress typically depends on *pivoting* between static inspection, dynamic observation, and hypothesis-driven scripting rather than exhausting any one in isolation [10, 53]. The Tier 3 represents each run as a sequence of behavioral states and summarizes it with a small set of pre-determined and interpretable features.

State set and tagging. Each action is classified at two granularities (Table 2): a six-state coarse states over which we estimate a Markov model: TRIAGE, STATIC (disassembly, decompilation), DYNAMIC (debugging, tracing, execution), SCRIPT (building or running a solver), VALIDATE (flag submission), and OTHER and a more granular layer of 20 codes that are categorized under these states. The 20 fine-grained behavioral codes were developed by iterative coding of a pilot set of trajectories, then grouped under the six coarse states. A precedence-ordered, command-pattern tagger maps each action to a (fine, coarse) pair, first match winning, so a compound shell line is assigned a single dominant intent; the ordering encodes specificity, so “gdb ./bin” is DYNAMIC, “file ./bin” is TRIAGE, and a bare “./bin” is DYNAMIC only when no analysis tool has already claimed the command. The harness’s tools (decompiler, disassembler, file-authoring) are mapped directly to STATIC, STATIC, and SCRIPT, so analysis through dedicated tools is not misattributed to OTHER; because the action space is a closed toolset. This results in unclassified residual and an unseen shell-command strings accounting to under 1% of actions on our corpus.

Per-run representation. We summarize each run as a first-order Markov chain over the six coarse interaction states in S . Given a run’s state sequence $s_1, \dots, s_T$, we count transitions only within a session. n_{ij} denotes the resulting within-session count of $i \rightarrow j$ transitions. From this matrix and the tagged action stream we read a small feature vector, each component capturing one behavior of interest. Refer to Appendix A.1 for a detailed explanation of each metric.

State	Meaning (representative tools)
TRIAGE	File/format reconnaissance (file, strings).
STATIC	Disassembly / decompilation (r2, objdump).
DYNAMIC	Debugging, tracing, executing file (gdb, ltrace).
SCRIPT	Building/running script (Python, z3/angr, patching).
VALIDATE	flag submission.
OTHER	Navigation, file reads, environment setup.

Table 2: The six coarse behavioral states. The tagger assigns a pair of coarse fine-grained codes.

4.7 Judge Validation

REVEREE’s scoring is deterministic when the presence of the evidence is the accomplishment. This applies to S1, S6, and S7 (decided by Channel A rules), and S2 (which is decided by Channel A when the tool output contains structural markers, and falls back to the judge otherwise). The LLM judge is consulted for the comprehension/insight stages S3, S4a, S4b, and S5: the process is outcome-blinded, and must cite a trajectory span for every verdict (Section 4.4). To ensure the LLM Judge outcome is reliable, we validate the pipeline by measuring whether the judge’s comprehension verdicts match those of a human expert. Additionally, we audit the deterministic rules for false positives and negatives.

Human agreement study. We sampled 15 runs from the judged corpus, chosen at random subject to covering both benchmarks (8 NYU-CTF, 7 picoCTF), both outcomes (9 solved, 6 unsolved), and the study’s two model families (5 Claude-family, 6 GPT-family, and 4 Codex runs, spanning six models); the sample covers 12 unique challenges, three of which appear twice under different agents (Appendix B, Table 10). An author with eight years of reverse-engineering experience served as the annotator and independently labeled all eight stages for each run yielding 120 (run, stage) decisions. The coding scheme was the eight-stage schema itself: the annotator applied the stage definitions of Section 4.3 directly, and read the same inputs the judge receives: the agent run and the per-challenge solution brief. For S1, S2, S6, and S7, the human labels audit Channel A; for S3–S5 the labels are compared against the judge’s Channel B verdicts.

Results. Table 3 reports per-stage agreement. Overall agreement is 94.2% (113/120; 95% CI [88.4, 97.1]). On the deterministic stages the audit found no rule errors (60/60; pooled CI [94.0, 100]). On the judged comprehension stages, judge–human agreement is 88.3% (53/60; CI [77.8, 94.2]). For the insight stages it is: 93.3% for control-flow comprehension (S3) and algorithm recognition (S4a), 86.7% for algorithm naming (S4b), and 80.0% for constraint extraction (S5). All seven disagreements fall in the judged stages.

Disagreement analysis. All seven disagreements fall in the judged stages, and they are concentrated rather than scattered. For example, a single solved run accounts for four of the seven. In this run, the annotator credited S3, S4a, S4b, and

Stage	Agree	Rate	95% CI
S1	15/15	100.0%	[79.6, 100]
S2	15/15	100.0%	[79.6, 100]
S3	14/15	93.3%	[70.2, 98.8]
S4a	14/15	93.3%	[70.2, 98.8]
S4b	13/15	86.7%	[62.1, 96.3]
S5	12/15	80.0%	[54.8, 93.0]
S6	15/15	100.0%	[79.6, 100]
S7	15/15	100.0%	[79.6, 100]
Judged	53/60	88.3%	[77.8, 94.2]
Deterministic	60/60	100.0%	[94.0, 100]
Overall	113/120	94.2%	[88.4, 97.1]

Table 3: Human agreement with REVEREE’s per-stage verdicts on 15 runs (120 decisions). S1, S2, S6, and S7 audit the Channel A rules. S3–S5 are compared against the Channel-B LLM judge. Intervals are Wilson score intervals.

S5 while the judge credited none of them. Note that six of the seven disagreements happened when *the LLM judge was too conservative*: human annotator marked the stage reached while judge did not. The single LLM judge false positive is an S4b verdict on a solved run where the judge accepted an algorithm naming while the annotator did not.

Overall, the result confirms that REVEREE provides reliable evaluations.

5 Evaluation

In this section, we evaluate state-of-the-art LLM agents on reverse-engineering (RE) capture-the-flag challenges. We organize the evaluation around three research questions:

- RQ1:** How does the performance of an LLM agent vary across frontier models, prompting strategies, and configurations on RE tasks?
- RQ2:** What causes LLM agents to fail on RE tasks, and which behavioral signatures distinguish failure modes?
- RQ3:** Do LLM agents solve RE challenges through memorization and surface-features, or genuine analysis?

5.1 Experimental Setup

Models. We evaluate nine frontier models from the two leading families, spanning different tiers/sizes and versions within each family (Table 4) while holding the agent framework (D-CIPHER [52]) fixed. Codex is a code-specialised variant included as a point of contrast.

Prompting strategies. We compare four widely used prompting strategies, holding the harness and tools fixed so that only the prompt differs. *Vanilla*, the default D-CIPHER prompt left unchanged: the agent is given the task and the available

Model	Family	Size	Version
haiku	Claude	Small	4.5
sonnet	Claude	Medium	4.5
opus 4.5	Claude	Large	4.5
opus 4.7	Claude	Large	4.7
nano	gpt5	Small	5.0
mini	gpt5	Medium	5.0
gpt5	gpt5	Large	5.0
gpt5.4	gpt5	Large	5.4
codex	gpt5	Large	5.3

Table 4: The nine evaluated models and the axes varied within each family. *Size* is the capability tier and *version* is the model release.

tools and acts directly, with no imposed reasoning format and no cross-step memory (baseline). *Ledger* carries a persistent scratchpad of state across steps: the planner is instructed to maintain a structured ledger of established facts and open hypotheses in its reasoning output [35], but no harness-level enforcement is imposed. *ReAct* [65] interleaves explicit reasoning with actions: the agent alternates between a natural-language thought about the current state and a tool call, so each action is conditioned on the preceding reasoning and each observation feeds the next thought. *Reflexion* [46] adds a verbal self-critique after a failed attempt: the agent reflects on why the attempt failed and carries that reflection forward as guidance for the next try. The four *multi-strategy* models (opus [4], codex [14], gpt5 [36], gpt5.4 [36]) are run under all four strategies; the remaining five are run under vanilla only. **Challenges.** We evaluate on 88 RE challenges from two sources. Intercode-CTF [64] which collects challenges from PicoCTF and NYU-CTF [42] and they contribute 27 and 61 challenges, respectively.

Scoring with REVEREE. Each trajectory is scored by REVEREE against an eight-stage RE schema (Section 4.3). We report three milestone metrics: *solve rate*, the fraction of runs reaching S7; *stage coverage*, the fraction of stages a run reaches which grants partial credit for how far the RE process got; and *progress depth*, the longest unbroken chain of consecutive stages. Additionally, we extract each run’s *exit reason* directly from the termination log. We also generate a vector of behavioral features over six coarse states from REVEREE’s Tier-3 (Section 4.6).

5.2 RQ1: Performance Across Configurations

5.2.1 Models

Table 5 reports per-model performance under vanilla prompting. The ranking by solve rate and the ranking by partial progress (stage coverage) disagree. Opus 4.7 has the highest solve rate (0.624). Codex, in contrast, has the *lowest* solve rate (0.447) but the *highest* stage coverage (0.621). Additionally, at \$0.86 per solve it is roughly 2.5× cheaper than any other

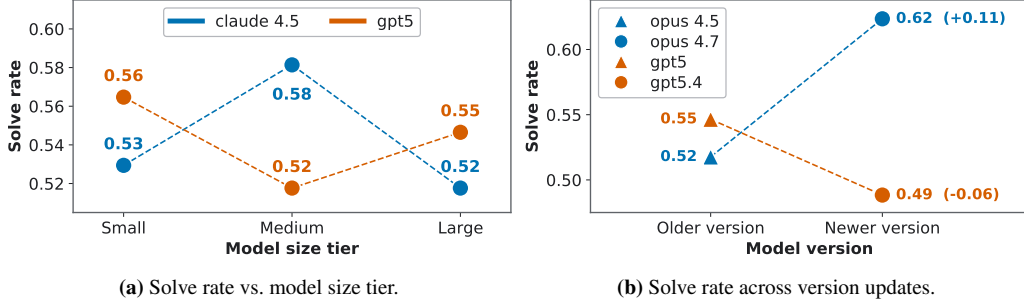


Figure 3: Within-family model comparison (a) Mean solve rate for each family (claude 4.5 in blue, gpt5 in orange) across three size tiers (small/medium/large). (b) Mean solve rate of each family’s older vs. newer version (marker shape denotes version), with the per-model change annotated. Solve rate is the fraction of runs reaching stage S7, pooled over the full dataset.

Model	Solve	Stage	\$/solve
opus 4.7	0.624	0.563	2.11
sonnet	0.581	0.613	2.63
nano	0.565	0.600	2.39
gpt5	0.547	0.587	2.78
haiku	0.529	0.591	2.52
opus 4.5	0.518	0.535	2.40
mini	0.518	0.568	2.59
gpt5.4	0.488	0.562	2.87
codex	0.447	0.621	0.86

Table 5: Per-model performance under vanilla prompting, sorted by solve rate. Cost is USD (\$) per solved challenge. Bold marks the best value in each column.

model. Leveraging the milestone schema used by REVEREE (Section 4.3), this means codex advances furthest through the RE process but converts that progress into a captured flag least often. A manual review of a subsample of codex’s trajectories corroborates the pattern: on the cases we examined, codex carries out the mechanical steps of RE well. It generates working scripts and uses the analysis tools effectively, but stalls before turning that work into a flag. The profile is consistent with a code-specialized model: strong at the individual operations RE requires (e.g., calling tools, writing, and running scripts), but is not able to reliably integrate their results into a validated flag. We offer this as an interpretation of the trajectories we examined rather than an established property of code-specialized models. Models also differ in how they work, e.g., tool reliance and reanalysis; Appendix A.2 reports these per-model fingerprints.

Model size. Neither family scales monotonically. Using a small / medium / large ladder of haiku / sonnet / opus 4.5 ² (Fig. 3a), Claude peaks at the *medium* tier: sonnet (0.581) is the strongest, and the large opus 4.5 (0.518) drops below it.

²For this experiment we use v4.5 because a size comparison must hold the version fixed, and the v4.7 has no medium (sonnet) or small (haiku) counterparts. For the other experiments, we use the stronger opus 4.7 as our main model for the Claude family.

This means that the largest model is the *weakest*. For gpt5: nano (0.565) matches/surpasses the full gpt5 (0.547), mini is the weakest of the three (0.518). These results show that within a version, size does not buy RE skill for either family.

Model versions. Fig. 3b shows that a newer model is not dependably a stronger one. The opus model has a large gain comparing version 4.5 to 4.7 (+0.106 solve overall, +0.138 on NYU-CTF). In the other direction, gpt5 outperforms gpt5.4 (−0.058 overall, −0.051 in NYU-CTF). We use REVEREE to localize the source of the regression and highlight two changes: First, gpt5.4 exhausts the budget cap on a larger share of its failures (73% vs. 69%) and reaches that cap in fewer executor turns (129 vs. 177) at comparable total cost. This translates to a higher per-step cost leading to less exploration. Second, its failures shift away from the round limit toward giving up (24% vs. 2%).

5.2.2 Time Analysis

Model speed. A run’s wall-clock time is set by how long the agent persists before it terminates (by solving, by giving up, or by exhausting the budget or round cap). The quickest-finishing configurations are opus 4.5 (1.9 min median), codex (3.2 min), opus 4.7 (3.4 min), and gpt5.4 (3.8 min): a set that spans the entire solve-rate range, from the strongest solver (opus 4.7) to the weakest (codex), and that is dominated by large opus models rather than small ones. What unites them is decisiveness not accuracy or size. The slowest runs come from gpt5 (12.5 min), nano (11.1 min), and haiku (10.9 min), a group of mixed size that includes the full-scale gpt5; these because they *grind*, making disorganized efforts to reach the solution (Fig. 4a). Counter to the intuition that smaller models are faster, the quickest agents here are the largest.

Solves vs. time relationship. When an agent solves a challenge, it exits faster than when it is unable to solve a challenge. Median time-to-solve is under a minute for the strongest configurations. Across all models, 70% of solves land within 2 minutes and 85% within 5 minutes. The wall-clock is therefore spent overwhelmingly on failures, which run 6–8×

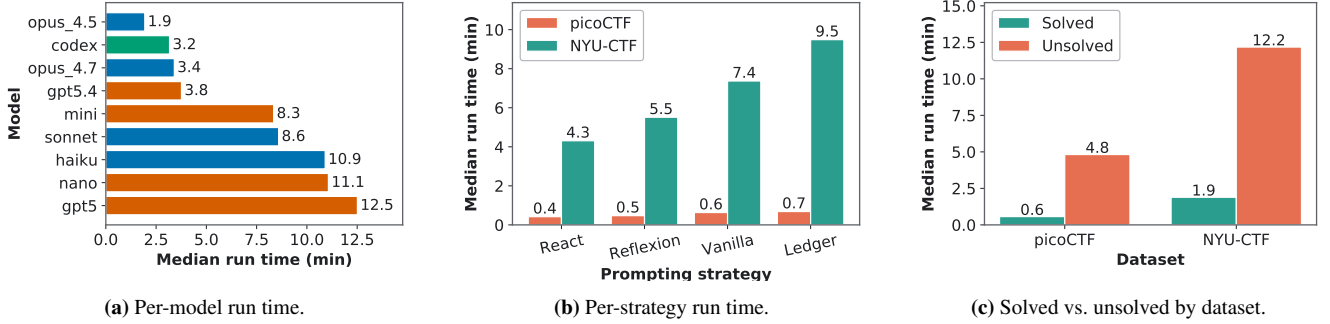


Figure 4: Wall-clock run time. (a) Median run time (minutes) per model on the vanilla strategy. (b) Median run time per prompting strategy, over the four multi-strategy models. (c) Median run time for solved vs. unsolved runs, grouped by dataset.

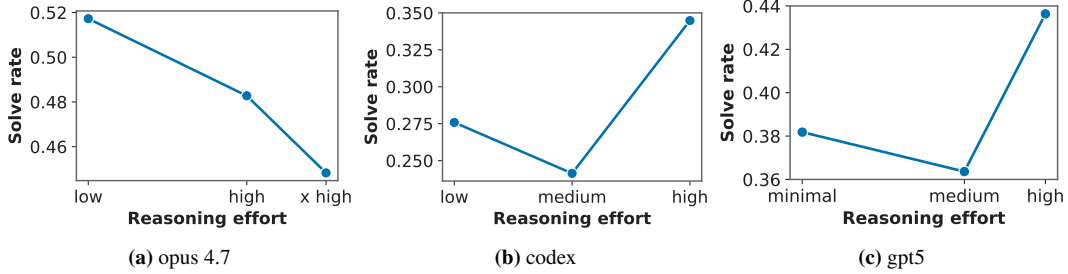


Figure 5: Each panel plots solve rate on NYU-CTF challenges across that model’s reasoning-effort levels.

longer than solves (Fig. 4c). On NYU-CTF, a solved run takes a median 1.9 min against 12.2 min for an unsolved one. PicoCTF has a similar pattern solved runs take a median of 0.6 min against 4.8 min for an unsolved one.

Prompting strategy speed. As shown in Fig. 4b, the reasoning-led ReAct (2.2 min) and Reflexion (2.6 min) are fastest and the memory-heavy Ledger (6.1 min) slowest, with Vanilla (4.1 min) between them; but the difference is small on the easier picoCTF (all 0.4–0.7 min) and is more noticeable on NYU-CTF (4.3–9.5 min). The dataset gap is an order of magnitude *within every strategy* (10–14×).

5.2.3 Prompting Strategies

Whether a prompting strategy helps depends on how hard the target is. On the saturated picoCTF the choice is immaterial: all four strategies solve between 0.852 and 0.880, a 2.8-point spread (Table 6). On the harder NYU-CTF the two reasoning-based strategies perform better. In the best performing model, opus, ReAct reaches 0.5 and Reflexion 0.552. This is a gain of +3.5% and +14% over vanilla. Other models show a performance gain between +19% and +35% over vanilla. Between the two reasoning-based strategies it becomes model-dependent (Table 6): ReAct shows better results for the gpt family (codex 0.362, gpt5 0.466, gpt5.4 0.458), while ledger and Reflexion (both 0.552) are the highest performers for opus. These experiments show that for more difficult challenges reasoning-based prompting strategies perform better

Model	Vanilla	Ledger	ReAct	Reflexion
<i>picoCTF</i>				
opus 4.7	0.926 / 0.593	0.926 / 0.644	0.926 / 0.542	0.926 / 0.532
codex	0.815 / 0.708	0.815 / 0.662	0.778 / 0.718	0.778 / 0.685
gpt5	0.889 / 0.694	0.926 / 0.579	0.889 / 0.611	0.889 / 0.620
gpt5.4	0.815 / 0.648	0.852 / 0.620	0.815 / 0.708	0.815 / 0.722
<i>NYU-CTF</i>				
opus 4.7	0.483 / 0.550	0.552 / 0.623	0.500 / 0.567	0.552 / 0.586
codex	0.276 / 0.580	0.310 / 0.547	0.362 / 0.573	0.316 / 0.570
gpt5	0.390 / 0.538	0.339 / 0.492	0.466 / 0.627	0.431 / 0.623
gpt5.4	0.339 / 0.523	0.271 / 0.483	0.458 / 0.714	0.373 / 0.631

Table 6: Solve rate/stage coverage for the four multi-strategy models under each prompting strategy, *per dataset*. **Bold** marks each model’s best solve rate.

but no single strategy dominates. The strategies that help do so by changing what the agent does, validating more often and re-analyzing the binary less (Appendix A.4); Section 5.3.2 makes this precise at the level of success and failure.

Prompting strategy solve overlap. Strategies solve mostly the same cases: out of the cases solved by *any* strategy, 72% are solved by all four and only 10% by exactly one. Counting unique contributions, ReAct solves 10 cases that no other strategy solves, against five for ledger, five for Reflexion, and two for Vanilla. This shows that ReAct is not only the best on average (for NYU-CTF) but also responsible for the most unique solves.

Configuration	Rescue	n (failures)
codex + persist	0.15	47
codex + persist + no giveup	0.10	41
opus 4.7 / budget	0.14	28
gpt5 / budget	0.10	29
gpt5 / rounds	0.11	9

Table 7: Rescue rate: fraction of same-strategy baseline failures of the targeted exit type solved after enabling each knob.

5.2.4 Reasoning Effort

Finally we vary the agent’s reasoning-effort setting on a matched subset of challenges (Fig. 5). *The effect is non-monotonic and model-specific.* Opus declines monotonically as effort increases: 0.517 with reasoning low, 0.483 at high (default setting), and 0.448 at extra-high. On this subset, the best results are achieved when reasoning effort is set to low. Codex and gpt5 are instead U-shaped: the medium setting is *worse* than the baseline and only the highest setting improves on it (codex from 0.241 to 0.345, gpt5 from 0.364 to 0.436).

INSIGHT. The base model is the primary determinant of performance. Within a family, neither a larger size tier nor a newer version reliably improves RE performance, and neither higher cost per solve nor longer run time indicates more thorough analysis; long runs are failed runs that persist without progress. Additionally, increasing reasoning effort is not monotonically beneficial. Prompting benefits are dataset-dependent: negligible on easy challenges and beneficial on harder ones.

5.3 RQ2: Failure Analysis and Signatures

RQ1 establishes *how often* agents succeed. We now use REVEREE to ask *why and where they fail, what* separates a successful run from a failed one, and whether targeted interventions help.

5.3.1 Failure Modes

Why agents fail. Across all failed runs ($n=852$), the most common termination is budget exhaustion (50%), followed by giving up (25%), hitting the planner-round cap (19%), tool errors (4%), and being killed/timeout (2%). The composition differs by model (Fig. 6). Codex *gives up* in 98% of its failures, i.e., it stops rather than exhausting its budget. The Claude models are limited by budget instead (opus 85%, sonnet 95%), haiku hits the planner round cap in 45% of its failures, and opus 4.5 is the most error-prone (37%). The gpt5 models split between budget and the planner round cap (gpt5 69% and 26%) or budget and giving up (gpt5.4 73% and 24%).

Where agents fail. Splitting each dataset’s funnel into solved versus unsolved runs (Table 8) localizes the stage where the two diverge, i.e., the point at which an agent fails. On the

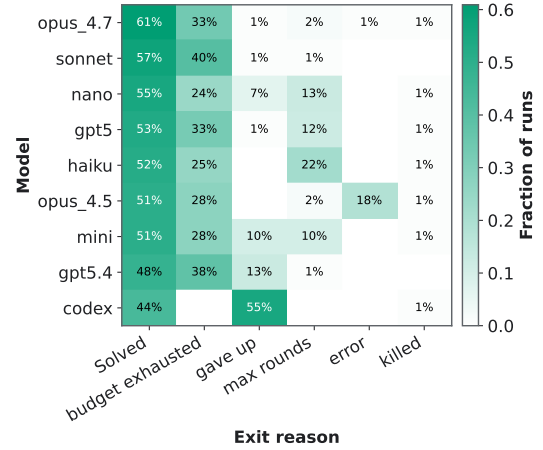


Figure 6: Exit-reason composition per model, pooled across all challenges. Each cell reports the fraction of that model’s runs ending in that exit reason.

NYU-CTF dataset, the two trajectories are similar through triage, surface mapping, and control-flow recovery (S1–S3) and then split at *algorithm identification* (S4a), reached by 0.76 of solved runs but only 0.46 of unsolved ones. Solved runs continue to surpass unsolved runs through constraint extraction (S5: 0.49 vs. 0.19) and solving (S6: 0.81 vs. 0.54). The bottleneck is therefore *comprehension*, not early reconnaissance or final scripting: failed agents recover the high-level structure of the binary but cannot *comprehend* the details needed to drive it to a flag. The picoCTF dataset inverts this trend: the unsolved line sits *above* the solved one at nearly every analysis stage (S4a 0.95 vs. 0.77, S6 0.92 vs. 0.71). This might be happening because picoCTF is an easier benchmark where most challenges are solved without a full reverse (skipping stages) of the binary. This is further corroborated with low progress depth for solved challenges (0.16). This is also consistent with our finding that models reproduce picoCTF flags from the challenge description alone (Section 5.4.1). The behavioral profiles match this divergence: picoCTF play is one-shot and static, whereas NYU-CTF is an iterative, dynamic grind (Appendix A.3)

Can configurations rescue failures. Having diagnosed the failures, we test whether the persistence, budget, and planner-round knobs recover them. For each knob we report the *rescue rate*: the fraction of a baseline’s same-strategy failures that the modified configuration now solves (Table 7). The interventions are well-matched to the diagnosis: a no-give-up prompt for the give-up-prone codex, a larger budget for the budget-bound Claude models. Rescue rates range from only 10–15% of failures. Budget increases the budget allocation per challenge from \$3 to \$5 and rescues 0.14 of opus’s vanilla failures; persistence rescues 0.10–0.15 of codex’s give-up failures. Raising the planner rounds cap from 30 to 50 helps only vanilla gpt5 (0.11). Raising any single allowance rarely

Dataset	Outcome	Depth	Stage	S1	S2	S3	S4a	S4b	S5	S6	S7
picoCTF	solved	0.160	0.650	0.99	0.06	0.67	0.77	0.41	0.59	0.71	1.00
	unsolved	0.329	0.693	1.00	0.36	0.82	0.95	0.69	0.80	0.92	0.00
NYU-CTF	solved	0.387	0.710	1.00	0.64	0.54	0.76	0.44	0.49	0.81	1.00
	unsolved	0.362	0.471	0.99	0.82	0.54	0.46	0.21	0.19	0.54	0.00

Table 8: split by outcome: RE-stage reach for solved vs. unsolved runs, per dataset.

converts a failure into a solve (across budget, rounds, and give-up knobs). The unrescued runs either re-exhaust the larger allowance or shift to the next binding limit: give-up→rounds for codex, rounds→budget for gpt5. This can allude to hitting a competence wall not resource caps.

5.3.2 Failure Signatures

We next ask what distinguishes a successful run from a failed one, using REVEREE’s Tier-3 features on the merged NYU-CTF set. We summarize by the results by computing the Cliff value as solved minus failed, so a positive sign marks a behavior enriched in successful runs. We exclude picoCTF from this comparison: its challenges are mostly easy and source-based, so a representative run is short, and only a few atypically long runs carry enough behavioral signal to profile.

What separates success from failure. One behavioral contrast holds across all models with varied prompting strategies, versions, sizes, and families. At the level of individual features, successful runs are distinguished by the *productive pivot*: moving from analysis into building a solver (Cliff’s [17] $\delta = +0.22$ to $+0.38$). Additionally, by the time spent scripting ($+0.09$ to $+0.34$) and the pivot rate ($+0.04$ to $+0.23$). On the other hand, failed runs are instead marked by churn/being stuck: command repetition ($\delta = -0.36$ to -0.48), repeated whole-program re-analysis (-0.21 to -0.26), and time lost to dynamic execution (-0.16 to -0.30). The same distinction appears as concrete action sequences. The patterns consistently occurring in successful runs are all pivots into scripting: STATIC→SCRIPT, STATIC→STATIC→SCRIPT, and SCRIPT→SCRIPT. Failed runs, in contrast, loop on re-reading and triage: STATIC→TRIAGE, STATIC→STATIC, and STATIC→STATIC→STATIC.

INSIGHT. How an agent fails depends on the model, not only the challenge. On hard challenges the bottleneck is comprehension; on easy ones, solved runs skip analysis resulting in shortcut solves. Raising allowances rescues few failures, pointing to a competence wall. Success and failure also have a consistent shape across models and strategies: successful runs pivot from analysis into building a solver, whereas failed runs re-analyze without advancing. Strategies that help do so by inducing this behavior: more validation, less re-analysis.

5.4 RQ3: Reasoning vs. Memorization

Public CTF challenges and their walkthroughs are available online and thus are likely included in the training data of models tested in Section 5.1. We therefore ask whether the competence measured for the models mentioned above is genuine reverse engineering or sensitivity to memorized public benchmarks. We hold the underlying RE task constant and perturb only the features that a memorizing or shortcut-reliant model would depend on, separating two channels: *flag recall* and *surface mutations*.

5.4.1 Flag Recall

We first ask each model to produce a challenge’s flag given only its description, *without providing the binary or any other files/tools*. In this case, it should be almost impossible for the model to solve the challenges without memorizing the solutions from its training data. We measure both *exact matches* of the flag and *partial matches* of the flag (i.e., the flag given by the model shares at least four consecutive characters with the real flag). PicoCTF results indicate memorization of the models. For example, Opus 4.7 reproduces 7.4% of picoCTF’s exact, complete flags (25.9% partial flags), without accessing the binaries at all. Gpt5 and codex reach 25.9% and 18.5% partial recall with no exact matches. On the NYU-CTF dataset the recall is near zero (at most 3.3% partial and no exact matches). The result indicates that picoCTF suffers more from model memorization.

5.4.2 Surface Mutations

We measure the solve-rate drop from surface perturbation alone, scoring each model on the challenges it solved at baseline. For each model we choose the samples that were solved by the baseline (vanilla), apply the perturbations to these samples and then rerun the experiments on the perturbed samples without changing anything else. This allows us to measure/isolate the solve-rate drop from surface perturbation alone. These perturbations, including renaming the challenges and paraphrasing the question description, are done manually and do not change the semantics of the challenge itself.

Table 9 reports solve rate and stage coverage, original→perturbed, split by dataset. On picoCTF, renaming and paraphrasing change solve rate by at most 5 points for any model. On NYU-CTF, solve rate drops

Pert.	Dataset	Model	Solve (o→p)	Stage (o→p)
Name	picoCTF	opus 4.7	1.00 → 0.96	0.59 → 0.54
		codex	1.00 → 0.95	0.72 → 0.64
		gpt5	1.00 → 0.95	0.71 → 0.59
	NYU-CTF	opus 4.7	1.00 → 0.92	0.69 → 0.63
		codex	1.00 → 0.81	0.75 → 0.75
		gpt5	1.00 → 0.70	0.70 → 0.59
Desc.	picoCTF	opus 4.7	1.00 → 1.00	0.59 → 0.56
		codex	1.00 → 0.95	0.72 → 0.68
		gpt5	1.00 → 0.95	0.71 → 0.66
	NYU-CTF	opus 4.7	1.00 → 0.92	0.69 → 0.68
		codex	1.00 → 0.87	0.75 → 0.60
		gpt5	1.00 → 0.75	0.70 → 0.61

Table 9: Effect of perturbing the name and description split by dataset: solve rate and stage coverage, original → perturbed, each model scored on its own solved baseline set. A decline indicates reliance on the perturbed cue.

7.7–30.0 points under the same two conditions: opus 4.7 falls 7.7 points under both; codex falls 18.8 points (renamed) and 12.5 points (paraphrased); gpt5 falls 30.0 points (renamed) and 25.0 points (paraphrased). Stage coverage shows the same split: near-zero change on picoCTF, larger declines on NYU-CTF except codex’s renamed-condition stage coverage. This shows that perturbations have an effect on the model’s ability to solve challenges, but the majority of the solves were still maintained. The result indicates that while the memorization effect exists, there is still a level of genuine reasoning to solve these RE tasks.

INSIGHT. Memorization is present and benchmark-dependent: picoCTF flags are recalled from descriptions alone; NYU-CTF shows minimal recall. Agents also lean on surface cues, but most solves survive perturbation. This means genuine analysis coexists with memorization, rather than being replaced by it. To use easy, public benchmarks, contamination-controlled evaluation is required.

6 Related Work

LLM agents for CTF and offensive security. Autonomous CTF agents are evaluated almost entirely by *solve rate*: NYU-CTF [42], Cybench [68], picoCTF [64], EnIGMA [1], D-CIPHER [52], and plain-agent baselines [51] all rank systems by the fraction of challenges whose flag is captured. EnIGMA equips a single agent with interactive terminal tools [1]; PentestGPT traces failure to context loss over long runs [19]; D-CIPHER adds a planner–executor multi-agent harness [52]. We contribute no agent; instead, we propose a method that evaluates agents’ runs based on an RE-specific stage schema end-to-end. We hold the D-CIPHER harness fixed and vary

the base model, prompting strategy, and configurations.

Language models for reverse engineering. A parallel line targets individual RE primitives rather than end-to-end challenge solving. These includes tasks such as neural and LLM decompilation (LLM4Decompile [50], SLaDe [6], Idioms [20], DeGPT [24], Nova [27]), recovery of names, types, and structures from stripped binaries (DIRTY [15], ReSym [60], GenNm [62], SymLM [28], XFL [37]), function-name inference [26], GNN-based type inference TYGR [70], disassembly and instruction representation (XDA [38], Loadstar [7], D-ARM [66], PalmTree [29], jTrans [57]), summarization and deobfuscation (BinT5 [2], Bin2Summary [48], Beste et al. [8]), and benchmarks of LLM binary understanding [41]. We instead evaluate agents that must orchestrate a full toolchain across a challenge end-to-end. If these tools are incorporated into the toolchain the agent has access to, they become one stage in a longer trajectory we score.

Evaluating agents beyond solve rate. CTFJudge scores agent trajectories with an LLM judge against expert write-ups across all CTF categories, introducing a Competency Index for partial correctness [43]. This is done using generic competency dimensions in a single whole-trajectory LLM pass. REVEREE, on the other hand, focuses on RE challenges and evaluates agents’ runs based on an RE-specific stage schema. Additionally, it confines the LLM judge to the comprehension stages of a reverse-engineering workflow. It requires the LLM judge to cite evidence for every award, blinds the judge to the outcome, and is validated against a human rater. Mechanical stages are evaluated deterministically. Another differentiating factor is the behavioral profiling provided by REVEREE. The broader code-agent literature supplies the precedent for judging trajectories at scale (SWE-agent [63]; Cemri et al.’s MAST taxonomy of multi-agent failures [11]); we adopt standard cautions on LLM-as-judge reliability [58, 69].

Failure analysis. Nishizaka et al. qualitatively attribute failures on 24 “crackmes” across three commercial agents to four weaknesses: training bias, over-trust in observations, context limitation, and plan persistence. Their analysis uses a three-stage (Observe–Comprehend–Plan) loop and qualitative labeling. Two evaluators hand-tag which protection exposes which weakness [32]. REVEREE differs in granularity, scale, and output. REVEREE computes partial solves and generates a behavioral profile (*output*) based on an eight-stage RE schema (*granularity*) across 88 challenges and nine models and four prompting strategies (*scale*). The nine-model, four-strategy scale is what licenses an invariance claim that [32]’s three-agent study cannot make, and the behavioral tier reports the run-level *dynamics* of failure not only its cognitive cause.

RE reasoning and memorization. Because CTF write-ups leak into pre-training, solve rate conflates capability with recall. A common mitigation perturbs benchmark items with semantics-preserving transformations and reads the drop as a contamination signal. Prior work suggests generic LLM-benchmark mutation methods that do not apply directly to

RE [39, 40], and they expose a fidelity–resistance tradeoff in which heavy rewrites distort the task [49]. An RE-specific precedent EnIGMA [1], does not perturb the challenge at all: it flags leakage *post hoc* from agent runs in two ways: 1) by detecting runs that submit the flag in a single step or without analysis and 2) by estimating contamination from a separate held-out set of post-cutoff challenges. The held-out comparison spans *different* challenges which can confound contamination with difficulty and the trajectory test registers only the most blatant recall. We instead adapt the perturbation idea to RE, mutating at the *challenge* level (renaming, paraphrasing), since the artifact an agent consumes is the compiled binary. This yields a difficulty-matched original-versus-perturbed pair.

7 Discussion

7.1 What Generalizes Beyond These Models

The rankings, solve rates, and costs above are a snapshot that the next model generation will overwrite; what survives is REVEREE and the structural regularities it exposes. As an evaluation framework, it asks where an agent fails, how it behaves, and whether success reflects analysis or recall, which will be applicable to future models. Three findings are claims about the task, not one model’s score, and should persist: a single solve rate conflates stage progress with flag capture (Section 5.2.1); the binding constraint is localizable and presently competence, not resources. Specifically, failures concentrate at comprehension stages, while budget, persistence, and round-limit interventions recover only 10–15% of them (Section 5.3.1); and productive and unproductive trajectories have a characteristic shape: validate-then-script versus re-read-and-loop. These patterns are replicated across all nine models (Section 5.3.2). The evaluation lessons strengthen with time: an easy benchmark hides genuine prompting effects (Section 5.2.3), and as write-ups leak ever further into pre-training (Section 5.4), a held-out split and a surface-mutation protocol become *more* necessary, not less. Re-run on the next generation, the framework reports not a new number but *what changed*: where the bottleneck moved, whether the signature holds, and whether scaffolding finally pays off.

7.2 Security-Specific Prompting Strategies

The four strategies we evaluate were built for other domains, where they deliver large gains: chain-of-thought lifts GSM8K accuracy by roughly forty points [59], and ReAct and Reflexion improve embodied task success by twenty to thirty-four points [46, 65]. On reverse engineering tasks, however, the same strategies move solve rate by under three points on the easy picoCTF challenges and by seven to twelve points on the harder NYU-CTF challenges, and overlap so heavily that 72% of solved cases are solved by all four (Sec-

tion 5.2.3). Prompting tuned for arithmetic or web navigation, in short, transfers poorly to RE. In other security domains, domain-specific *prompting* has been developed. For example, malware-scoped hierarchical prompting reaches 77% against 49.5% for a generic prompt [56], and vulnerability-semantics chain-of-thought raises macro-F1 from 3% to 36% [33]. Yet no RE-specific prompting strategies have been developed. A reasoning schema built for the comprehension stages, rather than a generic strategy imported as-is, is a concrete and largely unexplored direction.

7.3 Limitations and Future Work

Our work has a few limitations, each pointing to a future direction. First, every solve rate is single-attempt: one trajectory per (model, strategy, challenge), without pass@ k or resampling. Within a fixed budget, we prioritized breadth (9 models, 4 prompting strategies, 88 challenges) over repeated sampling. We anchor our claims on effects stable across the corpus: the comprehension bottleneck and the success–failure signature hold across all nine models. Estimating within-configuration variance under pass@ k on a stratified subset is a natural next step. Second, we instantiate REVEREE on a single harness, D-CIPHER. REVEREE is harness-agnostic, so replicating the analysis on other harnesses (e.g., ENIGMA [1]) would be a natural next step. Finally, our corpus spans two sources that behave differently: picoCTF and NYU CTF. This asymmetry is itself a finding. Scaling the evaluation with larger and more diverse challenge sets (e.g., harder binaries), and more difficulty-preserving perturbations would both sharpen REVEREE’s contamination signal.

8 Conclusion

This paper introduced REVEREE, a framework for evaluating LLM reverse-engineering agents beyond solve rate. REVEREE scores each run at three tiers, solve rate, milestone progress through an eight-stage RE schema, and a behavioral profile of the agent’s actions, and pairs them with a memorization probe. Applying it to nine frontier models under four prompting strategies on 88 picoCTF and NYU-CTF challenges, we found that the base model dominates performance and that larger, newer, or costlier models are not reliably stronger. We also found that failures concentrate at the comprehension stages and reflect a competence limit rather than a resource limit, while successful runs share a behavioral signature that failed runs lack. Finally, regarding memorization, models recall picoCTF flags from challenge descriptions alone whereas NYU-CTF shows near zero recall, and most solves survive surface perturbation, so analysis coexists with memorization rather than being replaced by it. The lasting contribution is the methodology: REVEREE can be applied to future agents and its diagnostic questions grow more relevant as contamination spreads. We release REVEREE to support future work.

References

- [1] Talor Abramovich, Meet Udeshi, Minghao Shao, Kilian Lieret, Haoran Xi, Kimberly Milner, Sofija Jancheska, John Yang, Carlos E. Jimenez, Farshad Khorrami, Prashanth Krishnamurthy, Brendan Dolan-Gavitt, Muhammad Shafique, Karthik R. Narasimhan, Ramesh Karri, and Ofir Press. EnIGMA: Interactive tools substantially assist LM agents in finding security vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [2] Ali Al-Kaswan, Toufique Ahmed, Maliheh Izadi, Anand Ashok Sawant, Premkumar Devanbu, and Arie van Deursen. Extending source code pre-trained language models to summarise decompiled binaries. In *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2023.
- [3] Sergi Alvarez and The Radare2 Project. Radare2. <https://rada.re/n/>, 2024.
- [4] Anthropic. System card: Claude opus 4.5. Technical report, Anthropic, 2025.
- [5] Andrey Anurin, Jonathan Ng, Kibo Schaffer, Jason Schreiber, and Esben Kran. Catastrophic cyber capabilities benchmark (3CB): Robustly evaluating LLM agent cyber offense capabilities, 2024.
- [6] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael F. P. O’Boyle. SLaDe: A portable small language model decompiler for optimized assembly. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2024.
- [7] Hadjer Benkraouda, Nirav Diwan, and Gang Wang. You can’t judge a binary by its header: Data-code separation for non-standard arm binaries using pseudo labels. *IEEE Symposium on Security and Privacy (SP)*, 2025.
- [8] David Beste, Grégoire Menguy, Hossein Hajipour, Mario Fritz, Antonio Emanuele Cinà, Sébastien Bardin, Thorsten Holz, Thorsten Eisenhofer, and Lea Schönherr. Exploring the potential of LLMs for code deobfuscation. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2025.
- [9] Adam R. Bryant. *Understanding How Reverse Engineers Make Sense of Programs from Assembly Language Representations*. PhD thesis, Air Force Institute of Technology, 2012.
- [10] Mariano Ceccato, Paolo Tonella, Cataldo Basile, Paolo Falcarin, Marco Torchiano, Bart Coppens, and Bjorn De Sutter. Understanding the behaviour of hackers while performing attack tasks in a professional setting and in a public challenge. *Empirical Software Engineering*, 24(1), 2019.
- [11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS), Datasets and Benchmarks Track*, 2025.
- [12] Sang Kil Cha, Thanassis Avgerinos, Alexandre Rebert, and David Brumley. Unleashing Mayhem on binary code. In *IEEE Symposium on Security and Privacy (S&P)*, 2012.
- [13] Peter Chapman, Jonathan Burket, and David Brumley. PicoCTF: A game-based computer security competition for high school students. In *USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE 14)*, 2014.
- [14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.
- [15] Qibin Chen, Jeremy Lacomis, Edward J. Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. Augmenting decompiler output with learned variable names and types. In *31st USENIX Security Symposium (USENIX Security)*, 2022.
- [16] Elliot J. Chikofsky and II Cross, James H. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17, January 1990.
- [17] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3), 1993.
- [18] Andrei Costin, Jonas Zaddach, Aurélien Francillon, and Davide Balzarotti. A Large-Scale analysis of the security of embedded firmwares. In *23rd USENIX Security Symposium (USENIX Security)*, 2014.
- [19] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security)*, 2024.
- [20] Luke Dramko, Claire Le Goues, and Edward J. Schwartz. Idioms: A simple and effective framework for turbocharging local neural decompilation with well-defined

- types. In *Proceedings of the 2026 Network and Distributed System Security Symposium (NDSS)*, 2026.
- [21] Eldad Eilam. *Reversing: Secrets of Reverse Engineering*. John Wiley & Sons, 2005.
 - [22] Luca Gioacchini, Marco Mellia, Idilio Drago, Alexander Delsanto, Giuseppe Siracusano, and Roberto Bifulco. AutoPenBench: Benchmarking generative agents for penetration testing. *arXiv preprint arXiv:2410.03225*, 2024.
 - [23] Hex-Rays. IDA Pro. <https://hex-rays.com/ida-pro/>, 2024.
 - [24] Peiwei Hu, Ruigang Liang, and Kai Chen. DeGPT: Optimizing decompiler output with LLM. In *Proceedings of the 2024 Network and Distributed System Security Symposium (NDSS)*, 2024.
 - [25] Zimo Ji et al. Measuring and augmenting large language models for solving capture-the-flag challenges. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.
 - [26] Linxi Jiang, Xin Jin, and Zhiqiang Lin. Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.
 - [27] Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, Xiangyu Zhang, and Petr Babkin. Nova: Generative language models for assembly code with hierarchical attention and contrastive learning. In *International Conference on Learning Representations (ICLR)*, 2025.
 - [28] Xin Jin, Kexin Pei, Jun Yeon Won, and Zhiqiang Lin. SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
 - [29] Xuezixiang Li, Yu Qu, and Heng Yin. PalmTree: Learning an assembly language model for instruction embedding. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.
 - [30] Lajos Muzsai, David Imolai, and András Lukács. HackSynth: LLM agent and evaluation framework for autonomous penetration testing. *arXiv preprint arXiv:2412.01778*, 2024.
 - [31] National Security Agency. Ghidra. <https://ghidra-sre.org/>, 2024.
 - [32] Ryutaro Nishizaka, Yudai Fujiwara, Takuya Shimizu, Kazushi Kato, and Yuichi Sugiyama. Towards LLM-resistant software protection: Agent failure patterns in CTF reverse engineering. In *Workshop on Binary Analysis Research (BAR), co-located with NDSS Symposium*, San Diego, CA, USA, February 2026.
 - [33] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230*, 2024.
 - [34] J. R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1997.
 - [35] Maxwell Nye, Anders Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
 - [36] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
 - [37] James Patrick-Evans, Moritz Dannehl, and Johannes Kinder. XFL: Naming functions in binaries with extreme multi-label learning. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.
 - [38] Kexin Pei, Jonas Guan, David Williams-King, Junfeng Yang, and Suman Jana. XDA: Accurate, robust disassembly with transfer learning. In *Proceedings of the 2021 Network and Distributed System Security Symposium (NDSS)*, 2021.
 - [39] Kun Qian, Shunji Wan, Claudia Tang, Youzhi Wang, Xuanming Zhang, Maximillian Chen, and Zhou Yu. VarBench: Robust language model benchmarking through dynamic variable perturbation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2024.
 - [40] Martin Riddell, Ansong Ni, and Arman Cohan. Quantifying contamination in evaluating code generation capabilities of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
 - [41] Xiuwei Shang, Shaoyin Cheng, Guoqiang Chen, Yanming Zhang, Li Hu, Xiao Yu, Gangyang Li, Weiming Zhang, and Nenghai Yu. How far have we gone in binary code understanding using large language models. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2024.

- [42] Minghao Shao, Sofija Jancheska, Meet Udeshi, Brendan Dolan-Gavitt, Haoran Xi, Kimberly Milner, Boyuan Chen, Max Yin, Siddharth Garg, Prashanth Krishnamurthy, Farshad Khorrani, Ramesh Karri, and Muhammad Shafique. NYU CTF bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [43] Minghao Shao, Nanda Rani, Kimberly Milner, Haoran Xi, Meet Udeshi, Saksham Aggarwal, Venkata Sai Charan Putrevu, Sandeep K. Shukla, Prashanth Krishnamurthy, Farshad Khorrani, Ramesh Karri, and Muhammad Shafique. Towards effective offensive security llm agents: Hyperparameter tuning, llm as a judge, and a lightweight ctf benchmark. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 29660–29668, 2026.
- [44] Minghao Shao, Haoran Xi, Nanda Rani, Meet Udeshi, Venkata Sai Charan Putrevu, Kimberly Milner, Brendan Dolan-Gavitt, Sandeep Kumar Shukla, Prashanth Krishnamurthy, Farshad Khorrani, Ramesh Karri, and Muhammad Shafique. Craken: Cybersecurity llm agent with knowledge-based execution. *ArXiv*, abs/2505.17107, 2025.
- [45] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. Llms in software security: A survey of vulnerability detection techniques and insights. 58(5), 2025.
- [46] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [47] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. SoK: (state of) the art of war: Offensive techniques in binary analysis. In *IEEE Symposium on Security and Privacy (S&P)*, 2016.
- [48] Zirui Song, Jiongyi Chen, and Kehuan Zhang. Bin2summary: Beyond function name prediction in stripped binaries with functionality-specific code embeddings. *Proceedings of the ACM on Software Engineering (PACMSE), FSE*, 2024.
- [49] Yifan Sun, Han Wang, Dongbai Li, Gang Wang, and Huan Zhang. The emperor’s new clothes in benchmarking? a rigorous examination of mitigation strategies for LLM benchmark data contamination. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [50] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [51] Rustem Turtayev, Artem Petrov, Dmitrii Volkov, and Denis Volk. Hacking CTFs with plain agents, 2024.
- [52] Meet Udeshi, Minghao Shao, Haoran Xi, Nanda Rani, Kimberly Milner, Sofija Jancheska, Brendan Dolan-Gavitt, Siddharth Garg, Ramesh Karri, and Muhammad Shafique. D-CIPHER: Dynamic collaborative intelligent multi-agent system with planner and heterogeneous executors for offensive security, 2025.
- [53] Daniel Votipka, Seth M. Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. An observational investigation of reverse engineers’ processes. In *29th USENIX Security Symposium (USENIX Security)*, 2020.
- [54] Daniel Votipka, Seth M. Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. An observational investigation of reverse engineers’ processes. In *29th USENIX Security Symposium (USENIX Security)*, 2020.
- [55] Jan Vykopal, Valdemar Švábenský, and Ee-Chien Chang. Benefits and pitfalls of using capture the flag games in university courses. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2020.
- [56] Brandon J. Walton, Mst Eshita Khatun, James M. Ghawaly, and Aisha Ali-Gombe. Exploring large language models for semantic analysis and categorization of android malware. In *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, 2024.
- [57] Hao Wang, Wenjie Qu, Gilad Katz, Wenyu Zhu, Zeyu Gao, Han Qiu, Jianwei Zhuge, and Chao Zhang. jTrans: Jump-aware transformer for binary code similarity detection. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022.
- [58] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhi-fang Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2024.

- [59] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022.
- [60] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.
- [61] Jiachen Xu, Jack W. Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *ArXiv*, abs/2403.01038, 2024.
- [62] Xiangzhe Xu, Zhuo Zhang, Zian Su, Ziyang Huang, Shiwei Feng, Yapeng Ye, Nan Jiang, Danning Xie, Siyuan Cheng, Lin Tan, and Xiangyu Zhang. Unleashing the power of generative model in recovering variable names from stripped binary. In *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.
- [63] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.
- [64] John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, *Datasets and Benchmarks Track*, 2023.
- [65] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [66] Yapeng Ye, Zhuo Zhang, Qingkai Shi, Yousra Aafer, and Xiangyu Zhang. D-ARM: Disassembling ARM binaries by lightweight superset instruction interpretation and graph modeling. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.
- [67] Miuyin Yong Wong, Matthew Landen, Manos Antonakakis, Douglas M. Blough, Elissa M. Redmiles, and Mustaque Ahamad. An inside look into the practice of malware analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.
- [68] Andy K. Zhang, Neil Perry, Riya Dulepet, Joey Ji, Celeste Menders, Justin W. Lin, Eliot Jones, Gashon Hussein, Samantha Liu, Donovan Julian Jasper, Pura Peetathawatchai, Ari Glenn, Vikram Sivashankar, Daniel Zamoshchin, Leo Glikbarg, Derek Askaryar, Haoxiang Yang, Aolin Zhang, Rishi K. Alluri, Nathan Tran, and et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. In *International Conference on Learning Representations*, 2025.
- [69] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, *Datasets and Benchmarks Track*, 2023.
- [70] Chang Zhu, Ziyang Li, Anton Xue, Ati Priya Bajaj, Wil Gibbs, Yibo Liu, Rajeer Alur, Tiffany Bao, Hanjun Dai, Adam Doupe, Mayur Naik, Yan Shoshitaishvili, Ruoyu Wang, and Aravind Machiry. TYGR: Type inference on stripped binaries using graph neural networks. In *33rd USENIX Security Symposium (USENIX Security)*, 2024.
- [71] Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. Training language model agents to find vulnerabilities with ctf-dojo. *ArXiv*, abs/2508.18370, 2025.
- [72] Yuwen Zou, Jia Liu, and Wenjun Fan. Ctfagent: An llm-powered agent for ctf challenge solving. *J. Inf. Secur. Appl.*, 96:104305, 2026.

A Behavioral Profiles: Definitions and Additional Results

This section provides extra details on the Tier 3 behavioral profile of Section 4.6 and related results. Appendix A.1 defines the components of the profile that Section 4.6 introduces only by name, all derived from the coarse-state transition matrix of a run. The remaining subsections slice the profile along the three factors varied in RQ1 (Section 5.2): Appendix A.2 reports per-model fingerprints, Appendix A.3 contrasts picoCTF and NYU-CTF play, and Appendix A.4 shows how the prompting strategies that help on NYU-CTF change what the agent does. None of these results alters an RQ1 conclusion; they explain *how* the differences reported there arise, and the last of them supplies the mechanism behind the success-versus-failure signature of Section 5.3.2.

A.1 Metric Definitions

The *static self-transition*, $n_{\text{STATIC,STATIC}} / \sum_k n_{\text{STATIC},k}$, measures how often the agent stays in static analysis from one step to the next, i.e., unproductive “grinding.” The *re-analysis count* is the number of times the agent re-runs whole-program analysis from scratch instead of reusing what an earlier agent invocation already produced. also reported as a rate normalized by the number of static actions. *Command repetition* is the fraction of issued shell commands that re-issue a previously seen, normalized command signature. The *pivot rate* is the fraction of transitions that change state, and the *productive pivot*, $n_{\text{STATIC,SCRIPT}} / \sum_k n_{\text{STATIC},k}$, is the empirical probability of moving into scripting conditional on leaving static analysis. The remaining components track how far the run moves beyond static reading: the static-to-dynamic action ratio and the position of the first dynamic action capture when and how much it turns from reading code to executing it, and two binary indicators record whether the run ever reached scripting (building a solver) or validation.

A.2 Model Behavior

Models also differ in *how* they work, not only in how often they succeed (Fig. 7). For example, models differ on how much tool reliance, the fraction of actions issued through decompilation, disassembly, or scripting rather than raw shell, they exhibit. Codex has a tool reliance value of 0.309, the most tool-reliant, while gpt5 at 0.144 is the most bash-centric. Additionally, we see differing profiles with respect to the coarse states reached by each model. Opus is the only model that reaches VALIDATE with significant frequency, i.e. that checks a candidate answer before submitting it. These fingerprints are stable across challenges.

A.3 Dataset Behavior

The behavioral profiles match this divergence (Fig. 9). PicoCTF play is one-shot and static: a static-to-dynamic action ratio of about 29, almost no re-analysis (0.02 passes per run), and the highest validation rate (0.14). NYU-CTF is an iterative, dynamic grind: a static-to-dynamic ratio of about 12, 4.42 re-analysis passes per run, and heavy scripting (0.90). At the model level, re-analysis count is the sharpest discriminator on NYU-CTF. gpt5 re-reads artifacts about 9.55 times per run against opus 4.7’s 1.39 (codex 2.20, gpt5.4 5.32) and codex again shows the deepest NYU-CTF progress depth (0.414) at the lowest solve rate (0.276).

A.4 Prompting Strategy Behavior

The strategies that help, help by changing what the agent does (Fig. 8). ReAct and Reflexion reach VALIDATE far more often than vanilla or ledger (0.13 and 0.15 versus 0.03) and re-analyze the binary about three times less often (1.0 and 1.1 passes per run versus 3.2 and 3.0). In other words, the strategies that win do so by checking answers and avoiding redundant re-analyzing, a connection we make precise at the level of success and failure in Section 5.3.2.

B Judge Validation Sample

Table 10 lists the 15 runs used in the human agreement study of Section 4.7, identified by dataset, outcome, agent, and challenge. The sample was drawn at random from the judged corpus subject to covering both benchmarks, both outcomes, and all three model families; the annotation procedure and results are reported in Section 4.7.

#	Dataset	Outcome	Agent	Challenge
1	NYU-CTF	unsolved	opus 4.7	2020q-rev-ebreezy
2	NYU-CTF	unsolved	codex	2020q-rev-ebreezy
3	NYU-CTF	unsolved	gpt5	2019f-rev-wrangly
4	NYU-CTF	unsolved	nano	2019f-rev-steady_counting
5	NYU-CTF	solved	opus 4.7	2023q-rev-whataxor
6	NYU-CTF	solved	sonnet	2023q-rev-rebug_2
7	NYU-CTF	solved	codex	2023q-rev-rebug_2
8	NYU-CTF	solved	codex	2021q-rev-checker
9	picoCTF	solved	opus 4.7	2025q-rev-10
10	picoCTF	solved	gpt5	2025q-rev-10
11	picoCTF	solved	sonnet	2025q-rev-52
12	picoCTF	solved	gpt5.4	2025q-rev-44
13	picoCTF	solved	gpt5.4	2025q-rev-46
14	picoCTF	unsolved	codex	2025q-rev-15
15	picoCTF	unsolved	gpt5	2025q-rev-43

Table 10: Runs in the human agreement sample: 15 runs over 12 unique challenges, chosen at random subject to covering both benchmarks, both outcomes, and all two agent families.

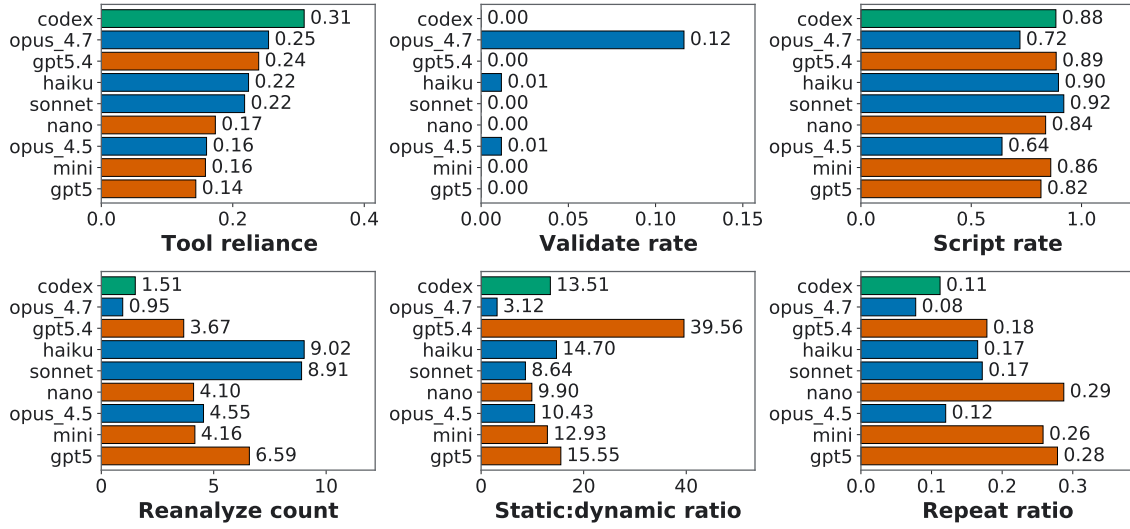


Figure 7: Behavioral fingerprints per model (vanilla). Panels report Tier-3 profile features averaged over challenges.

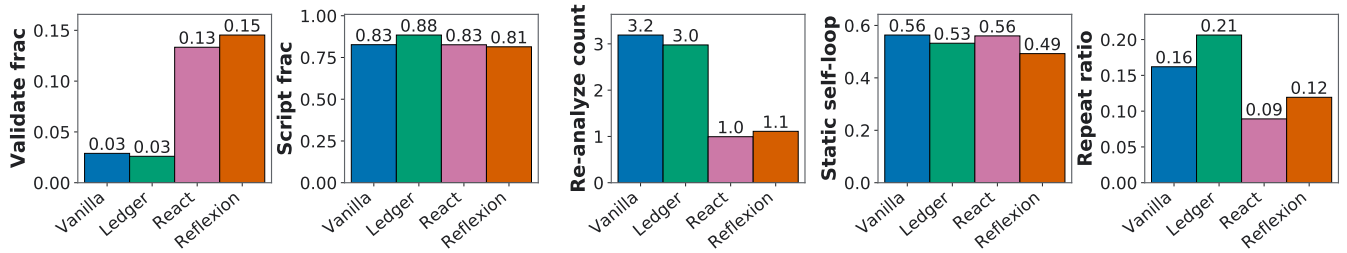


Figure 8: Behavioral profile by prompting strategy. Each panel reports one Tier-3 profile feature averaged over the four multi-strategy models.

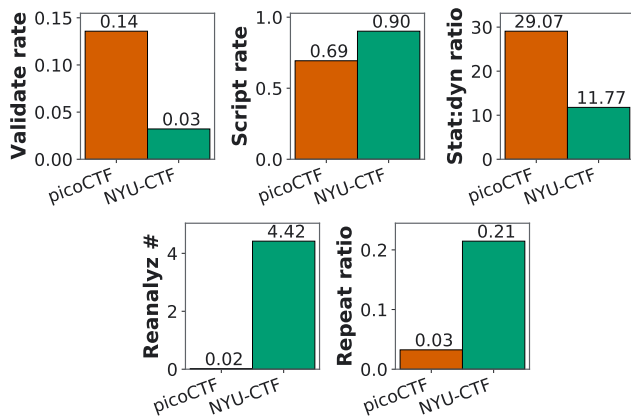


Figure 9: Behavioral profile per dataset. Each panel reports one Tier-3 profile feature, with one bar for picoCTF and one for NYU-CTF.